

Lembar Materi & Praktikum: Security API (CORS & Rate Limiter) serta Kontainerisasi dengan Docker & Docker Compose

Durasi: 3 Jam (180 Menit) **Level:** Advanced (Lanjut)

Praktikum ini melengkapi materi teori pada [20. secure-api.md](#). Anda akan dipandu untuk mengamankan REST API dari penyalahgunaan akses dan membungkus aplikasi agar siap dirilis ke server produksi pada proyek **Sistem Tiket Konser** (`go-tiket-konser`) dengan framework **Gin**. Kita akan belajar membatasi asal domain pemanggil menggunakan **CORS (Cross-Origin Resource Sharing)**, membatasi laju request per menit menggunakan **Rate Limiter**, menyusun kemasan aplikasi biner minimalis menggunakan **Multi-Stage Dockerfile**, serta mengorkestrasikan multi-container (App, Database, dan Redis Cache) menggunakan **Docker Compose**.

Sesi 1: Teori CORS dan Rate Limiter untuk Mengamankan API (45 Menit)

1. Cross-Origin Resource Sharing (CORS)

Secara default, browser web menerapkan kebijakan keamanan ketat yang disebut **Same-Origin Policy**. Kebijakan ini melarang aplikasi frontend yang berjalan di domain A (misal: `http://localhost:3000` atau `http://konsertiket.com`) untuk membaca data dari API backend yang berjalan di domain B (misal: `http://localhost:8080` atau `http://api.konsertiket.com`).

Jika frontend memaksa memanggil backend origin lain tanpa izin khusus, browser akan memotong respons dan memunculkan error CORS di konsol developer browser.

Solusinya adalah CORS Middleware: Backend harus mengirimkan header HTTP persetujuan khusus untuk memberi tahu browser bahwa domain frontend tersebut diperbolehkan melintasi batas origin.

- Header utama: `Access-Control-Allow-Origin` , `Access-Control-Allow-Methods` , `Access-Control-Allow-Headers` .

- **WARNING**
Di server produksi, **jangan pernah** menyetel `Access-Control-Allow-Origin: *` (wildcard/semua orang boleh mengakses) jika API tersebut bersifat privat atau membutuhkan kuki sesi / JWT kredensial. Selalu tentukan domain frontend terpercaya secara eksplisit.

2. Rate Limiter (Pembatasan Laju Akses)

Tanpa pembatasan laju akses, backend API Anda akan rentan terhadap berbagai serangan dunia maya:

- **Brute-Force Attacks:** Hacker berulang kali menembak endpoint login ribuan kali per menit untuk menebak password.
- **Scraping / Ticket Hoarding Bot:** Bot membeli seluruh tiket konser dalam hitungan detik (war tiket) dan menyingkirkan pembeli manusia yang asli.
- **DDoS (Distributed Denial of Service):** Request bertubi-tubi yang membanjiri server hingga RAM dan CPU habis dan sistem lumpuh (*server down*).

Algoritma Token Bucket: Salah satu algoritma rate limiting terpopuler. Server diasosiasikan memiliki wadah berisi sejumlah token. Setiap request masuk wajib mengambil 1 token. Jika wadah kosong, request ditolak dengan status HTTP **429 Too Many Requests**. Wadah token akan terisi kembali (*refill*) secara berkala sesuai konfigurasi (misalnya terisi 5 token setiap 1 detik).

Sesi 2: Teori & Mitigasi OWASP Top 10 2021 pada Backend Golang Gin (45 Menit)

Sebagai programmer backend Golang yang menggunakan framework Gin, memahami dan mengimplementasikan langkah-langkah pencegahan terhadap celah keamanan berdasarkan **OWASP Top 10 2021** adalah wajib hukumnya. Berikut adalah ringkasan klasifikasi celah keamanan tersebut beserta implementasi mitigasi konkretnya di Go/Gin:

1. A01:2021–Broken Access Control

- **Masalah:** Pengguna dapat mengakses resource milik orang lain karena verifikasi otorisasi yang tidak memadai (IDOR), atau mengakses endpoint admin tanpa hak akses level tinggi.
- **Anti-Pattern (Rentan):**

```
// Mengambil tiket hanya berdasarkan ID dari parameter tanpa mencocokkan kepemilikan user
r.GET("/tickets/:id", func(c *gin.Context) {
    ticketID := c.Param("id")
```

```

var ticket Ticket
db.First(&ticket, ticketID) // Kerentanan IDOR: user mana pun bisa membaca
ticket ID apa pun
c.JSON(200, ticket)
})

```

- **Mitigasi (Aman):** Gunakan middleware otentikasi JWT untuk menyuntikkan data user aktif (`userID`) ke konteks Gin, lalu pastikan resource yang diambil cocok dengan pemiliknya.

```

r.GET("/tickets/:id", middleware.AuthMiddleware(), func(c *gin.Context) {
    ticketID := c.Param("id")
    userID, _ := c.Get("userID") // Diambil dari claims JWT

    var ticket Ticket
    if err := db.Where("id = ? AND user_id = ?", ticketID,
userID).First(&ticket).Error; err != nil {
        c.JSON(403, gin.H{"error": "Akses ditolak atau resource tidak
ditemukan"})
        return
    }
    c.JSON(200, ticket)
})

```

2. A02:2021–Cryptographic Failures

- **Masalah:** Penyimpanan password berupa teks polos (cleartext), penggunaan algoritma hash kuno (MD5/SHA1), atau hardcoding kunci rahasia/JWT secret di dalam source code.
- **Anti-Pattern (Rentan):**

```

var jwtSecret = []byte("my_super_secret_key_123") // Hardcoded secret!

```

- **Mitigasi (Aman):**
 - Gunakan pustaka hashing modern seperti `bcrypt` untuk password.
 - Ambil secret key dari Environment Variables.

```

import (
    "os"
    "golang.org/x/crypto/bcrypt"
)

// Hashing password dengan Bcrypt
hashedBytes, _ := bcrypt.GenerateFromPassword([]byte(password),
bcrypt.DefaultCost)

// Membaca JWT Secret dari environment
var jwtSecret = []byte(os.Getenv("JWT_SECRET"))

```

3. A03:2021–Injection

- **Masalah:** SQL Injection (SQLi) yang terjadi akibat penggabungan string (string concatenation) secara mentah pada query database, atau Command Injection.
- **Anti–Pattern (Rentan):**

```
// Rentan SQL Injection karena menggunakan penggabungan string
query := fmt.Sprintf("SELECT * FROM users WHERE email = '%s'", emailInput)
db.Raw(query).Scan(&user)
```

- **Mitigasi (Aman):** Gunakan query terparameterisasi (*Parameterized Queries / Prepared Statements*). Pustaka ORM seperti GORM secara default melakukan hal ini.

```
// Menggunakan Parameterized Query bawaan GORM (Placeholder ?)
db.Where("email = ?", emailInput).First(&user)
```

4. A04:2021–Insecure Design

- **Masalah:** Kegagalan merancang arsitektur aplikasi yang aman (seperti tidak adanya limitasi transaksi, atau alur reset password yang mudah ditebak).
- **Mitigasi:** Implementasikan threat modeling saat perancangan, validasi alur bisnis, serta terapkan pertahanan berlapis (seperti CORS dan Rate Limiter) sejak fase desain awal.

5. A05:2021–Security Misconfiguration

- **Masalah:** Meninggalkan framework backend dalam mode debug di lingkungan produksi sehingga memunculkan log error verbose yang membeberkan informasi internal server ke publik.
- **Anti–Pattern (Rentan):** Menjalankan aplikasi di server produksi dengan default Gin mode (debug), yang akan mencetak seluruh route dan stack trace error ke response klien.
- **Mitigasi (Aman):**
 - Setel mode Gin menjadi `ReleaseMode` saat berjalan di produksi.
 - Pasang middleware recovery kustom untuk menangani panic tanpa membeberkan internal code.

```
func main() {
    if os.Getenv("APP_MODE") == "production" {
        gin.SetMode(gin.ReleaseMode)
    }
    r := gin.New()
    r.Use(gin.Recovery()) // Menangkap panic dan mengembalikan status 500 generik
}
```

6. A06:2021–Vulnerable and Outdated Components

- **Masalah:** Menggunakan modul pihak ketiga (dependency) yang memiliki kerentanan keamanan yang telah diketahui publik.
- **Mitigasi:**
 - Lakukan pemeriksaan kerentanan dependency secara berkala menggunakan tool resmi dari tim Go:

```
go install golang.org/x/vuln/cmd/govulncheck@latest
govulncheck ./...
```

- Selalu perbarui dependencies di berkas `go.mod` ke versi stabil terbaru (`go get -u`).

7. A07:2021–Identification and Authentication Failures

- **Masalah:** Autentikasi yang lemah (mengizinkan password super pendek), tidak membatasi percobaan login (*credential stuffing*), atau masa kedaluwarsa JWT token yang tidak masuk akal.
- **Mitigasi:**
 - Terapkan validasi kompleksitas password saat registrasi.
 - Batasi upaya login gagal menggunakan *Rate Limiter* khusus pada route `/api/login`.
 - Atur masa kedaluwarsa JWT token secukupnya (misal: 15 menit hingga 1 jam) dan manfaatkan Refresh Token.

8. A08:2021–Software and Data Integrity Failures

- **Masalah:** Menerima payload JSON yang tidak divalidasi dengan baik, atau melakukan deserialisasi dari sumber tidak terpercaya.
- **Mitigasi:** Gunakan Gin structural validation memanfaatkan pustaka validator bawaan Gin (`go-playground/validator`).

```
type RegisterInput struct {
    Email    string `json:"email" binding:"required,email"`
    Password string `json:"password" binding:"required,min=8"`
}

r.POST("/register", func(c *gin.Context) {
    var input RegisterInput
    if err := c.ShouldBindJSON(&input); err != nil {
        c.JSON(400, gin.H{"error": "Validasi input gagal: " + err.Error()})
        return
    }
})
```

9. A09:2021–Security Logging and Monitoring Failures

- **Masalah:** Tidak ada pencatatan logs untuk aktivitas krusial (seperti login gagal, transaksi besar), logs disimpan dalam teks polos yang mencatat kredensial/password, atau tidak dipantau secara real-time.
- **Mitigasi:**
 - Log setiap anomali keamanan (misalnya rate limit terlampaui atau gagal login beruntun).
 - Pastikan logs berformat terstruktur (JSON) menggunakan pustaka seperti `zap` atau `logrus` agar mudah di-parse oleh sistem SIEM (seperti Wazuh/ELK).
 - **PENTING:** Jangan pernah mencantumkan data sensitif (password, PIN, token JWT) di dalam log.

10. A10:2021–Server–Side Request Forgery (SSRF)

- **Masalah:** Server mengeksekusi request HTTP ke pihak ketiga berdasarkan URL/input yang diberikan oleh pengguna, memungkinkan penyerang mengakses server internal yang berada di balik firewall.
- **Anti–Pattern (Rentan):**

```
// Server mengunduh gambar berdasarkan URL bebas kiriman user
r.GET("/fetch-image", func(c *gin.Context) {
    imageUrl := c.Query("url")
    resp, _ := http.Get(imageUrl) // Kerentanan SSRF: penyerang bisa memasukkan
    URL "http://127.0.0.1:5432"
    io.Copy(c.Writer, resp.Body)
})
```

- **Mitigasi (Aman):**
 - Lakukan pencocokan URL input dengan daftar domain tepercaya (whitelist).
 - Cegah server melakukan koneksi keluar ke alamat IP privat/lokal (RFC 1918) seperti `127.0.0.1` , `10.0.0.0/8` , `172.16.0.0/12` , dan `192.168.0.0/16` .

Sesi 3: Teori Kontainerisasi dengan Docker dan Docker Compose (45 Menit)

1. Docker: Mengatasi Masalah "It Works on My Machine"

- **Docker Container** adalah teknologi virtualisasi tingkat sistem operasi (OS-level virtualization) yang membungkus kode program, runtime Go, dependencies, dan konfigurasi lingkungan ke dalam satu wadah terisolasi yang disebut **Container**.

- Berbeda dengan Virtual Machine (VM) yang memboyong seluruh kernel OS tamu (ukuran bergiga-giga dan boot lambat), Docker Container berjalan di atas kernel OS host yang sama, menjadikannya sangat ringan, hemat RAM, dan menyala dalam hitungan milidetik.

2. Multi-Stage Dockerfile di Go

Di Go, kita tidak membutuhkan kode sumber `.go` atau compiler Go di dalam server produksi. Kita hanya memerlukan biner hasil kompilasi. Kita memanfaatkan **Multi-Stage Build** pada Dockerfile untuk memotong ukuran Image Docker secara drastis:

- **Stage 1 (Build):** Menggunakan base image `golang:alpine` lengkap untuk mengunduh dependencies dan mengompilasi biner Linux Go.
- **Stage 2 (Runtime):** Menggunakan base image minimalis `alpine:latest` atau `scratch` yang hanya berukuran ~5MB, lalu menyalin biner dari Stage 1 ke dalamnya.
- *Hasil:* Ukuran Docker Image terpengkas dari yang semula ~800MB (jika menggunakan runtime Go penuh) menjadi hanya sekitar **~20MB saja!**

3. Docker Compose

Aplikasi riil selalu membutuhkan dependencies eksternal (multi-service), seperti database PostgreSQL dan in-memory cache Redis. Menjalankan image-image tersebut satu per satu secara manual sangat melelahkan dan rentan kesalahan konfigurasi jaringan.

Docker Compose menggunakan berkas deklaratif `docker-compose.yml` untuk mendefinisikan, mengonfigurasi, dan menjalankan multi-container tersebut secara bersamaan dalam jaringan terisolasi yang sama hanya dengan satu perintah tunggal:

```
docker compose up -d
```

Sesi 4: Laboratorium Praktikum Mandiri – Implementasi CORS dan Rate Limiter di Gin (45 Menit)

Pelajari langkah-langkah penulisan middleware keamanan di bawah ini untuk proyek konser Anda.

1. Membuat CORS Middleware Kustom: [middleware/cors.go](https://github.com/rsms/cors)

Buat berkas baru bernama `cors.go` di dalam folder `middleware/` untuk menyaring dan mengizinkan origin frontend:

```
package middleware
```

```

import (
    "net/http"

    "github.com/gin-gonic/gin"
)

// CorsMiddleware membatasi akses lintas domain dari aplikasi frontend terpercaya
func CorsMiddleware() gin.HandlerFunc {
    return func(c *gin.Context) {
        // Ubah origin ini dengan alamat domain frontend asli Anda saat
        produksi

        allowedOrigin := "http://localhost:3000"

        origin := c.Request.Header.Get("Origin")
        if origin == allowedOrigin {
            c.Writer.Header().Set("Access-Control-Allow-Origin",
allowedOrigin)

            c.Writer.Header().Set("Access-Control-Allow-Methods", "POST,
GET, OPTIONS, PUT, DELETE")
            c.Writer.Header().Set("Access-Control-Allow-Headers",
"Content-Type, Content-Length, Accept-Encoding, X-CSRF-Token, Authorization, accept,
origin, Cache-Control, X-Requested-With, X-API-Key")
            c.Writer.Header().Set("Access-Control-Expose-Headers",
"Content-Length")
            c.Writer.Header().Set("Access-Control-Allow-Credentials",
"true")
        }

        // Jika browser mengirimkan preflight OPTIONS request, segera potong
        siklus request
        if c.Request.Method == "OPTIONS" {
            c.AbortWithStatus(http.StatusNoContent)
            return
        }

        c.Next()
    }
}

```

2. Membuat Rate Limiter Middleware: [middleware/ratelimit.go](#)

Pertama, pasang pustaka rate limiter standar di terminal proyek Anda:

```
go get -u golang.org/x/time/rate
```

Buat berkas baru bernama `ratelimit.go` di bawah folder `middleware/` menggunakan algoritma Token Bucket:

```

package middleware

import (
    "net/http"
    "sync"

    "github.com/gin-gonic/gin"
    "golang.org/x/time/rate"
)

// IP-based Rate Limiter map untuk membatasi request per IP Address
var (
    limiters = make(map[string]*rate.Limiter)
    mu       sync.Mutex
)

// getLimiter mengembalikan limiter khusus untuk satu IP Address
func getLimiter(ip string) *rate.Limiter {
    mu.Lock()
    defer mu.Unlock()

    limiter, exists := limiters[ip]
    if !exists {
        // Batasan: Refill 5 token per detik, dengan kapasitas wadah maksimal
        10 token

        limiter = rate.NewLimiter(5, 10)
        limiters[ip] = limiter
    }

    return limiter
}

// RateLimiterMiddleware membatasi laju akses klien berdasarkan alamat IP mereka
func RateLimiterMiddleware() gin.HandlerFunc {
    return func(c *gin.Context) {
        ip := c.ClientIP()
        limiter := getLimiter(ip)

        // Ambil 1 token dari bucket. Jika gagal (bucket kosong), gagalkan
        request

        if !limiter.Allow() {
            c.AbortWithStatusJSON(http.StatusTooManyRequests, gin.H{
                "success": false,
                "message": "Terlalu banyak permintaan (Rate limit
                exceeded). Harap tunggu beberapa saat.",
            })
            return
        }

        c.Next()
    }
}

```

```
}  
}
```

Daftarkan kedua middleware ini pada file setup routing utama `routes/routes.go` :

```
func SetupRouter(db *gorm.DB) *gin.Engine {  
    r := gin.New()  
    r.Use(gin.Recovery())  
  
    // Terapkan CORS dan Rate Limiting secara global sebelum memetakan routing API  
    r.Use(middleware.CorsMiddleware())  
    r.Use(middleware.RateLimiterMiddleware())  
    r.Use(middleware.LoggerMiddleware())  
  
    // ... (setup endpoints) ...  
    return r  
}
```

Sesi 5: Laboratorium Praktikum Mandiri – Pembuatan Dockerfile & Docker Compose (45 Menit)

Kita akan mengemas seluruh infrastruktur backend agar siap dijalankan secara terpadu di server.

1. Menulis Berkas Dockerfile Multi-Stage: [Dockerfile](#)

Buat file bernama `Dockerfile` (tanpa ekstensi) di root folder proyek Anda:

```
# =====  
# STAGE 1: Build Stage (Mengompilasi source code)  
# =====  
FROM golang:1.22-alpine AS builder  
  
# Set working directory di dalam container  
WORKDIR /app  
  
# Salin go.mod dan go.sum terlebih dahulu untuk caching dependencies  
COPY go.mod go.sum ./  
RUN go mod download  
  
# Salin seluruh kode program  
COPY . .  
  
# Kompilasi aplikasi menjadi biner Linux statis (tanpa CGO agar mandiri)  
RUN CGO_ENABLED=0 GOOS=linux GOARCH=amd64 go build -ldflags="-w -s" -o go-tiket-konser main.go
```

```
# =====
# STAGE 2: Runtime Stage (Menjalankan berkas biner)
# =====
FROM alpine:latest

WORKDIR /app

# Salin biner hasil kompilasi dari builder stage
COPY --from=builder /app/go-tiket-konser .
# Salin berkas template lingkungan .env.example
COPY --from=builder /app/.env.example .env

# Ekspos port aplikasi
EXPOSE 8080

# Jalankan aplikasi biner
CMD ["/go-tiket-konser"]
```

2. Menulis Berkas Orkestrasi Multi-Container: [docker-compose.yml](#)

Buat file bernama `docker-compose.yml` di root folder proyek Anda untuk mendefinisikan kontainer web app, PostgreSQL database, dan Redis:

```
version: '3.8'

services:
  # 1. Database Service (PostgreSQL)
  db:
    image: postgres:15-alpine
    container_name: ticket_db
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: secret45
      POSTGRES_DB: eticketdb
    ports:
      - "5434:5432" # Ekspos port 5432 kontainer ke port 5434 host
    volumes:
      - pgdata:/var/lib/postgresql/data
    networks:
      - ticket_network

  # 2. Caching Service (Redis)
  cache:
    image: redis:7-alpine
    container_name: ticket_cache
    ports:
      - "6379:6379"
    networks:
      - ticket_network
```

```
# 3. Web Application Service (Backend Go)
web:
  build:
    context: .
    dockerfile: Dockerfile
  container_name: ticket_app
  ports:
    - "8080:8080"
  environment:
    APP_PORT: 8080
    APP_MODE: production
    DB_USER: postgres
    DB_PASS: secret45
    DB_HOST: db # Menggunakan nama service database sebagai host di internal
jaringan Docker
    DB_PORT: 5432 # Port internal container database
    DB_NAME: eticketdb
    JWT_SECRET: super_rahasia_konser_tiket_123!
  depends_on:
    - db
    - cache
  networks:
    - ticket_network

volumes:
  pgdata: # Menyimpan data database di disk host secara persisten agar tidak hilang
saat container mati

networks:
  ticket_network:
    driver: bridge
```

3. Perintah Utama Eksekusi Docker

Jalankan dan kelola container Anda menggunakan perintah-perintah sakti di bawah ini:

- **Menjalankan Seluruh Service di Latar Belakang:**

```
docker compose up --build -d
```

- **Memeriksa Status Container yang Sedang Aktif:**

```
docker compose ps
```

- **Membaca Log Output Real-time Aplikasi Backend:**

```
docker compose logs -f web
```

- Mematikan Seluruh Infrastruktur dan Menghapus Container:

```
docker compose down
```

Sesi 6: Tantangan Praktikum Mandiri (Tugas Mandiri)

Selesaikan tantangan arsitektur keamanan dan orkestrasi ini secara mandiri untuk menguasai kontainerisasi tingkat lanjut.

Tantangan 1: Rate Limiter Dinamis Berbasis IP Address Menggunakan Redis

Rate limiter memori lokal (golang.org/x/time/rate) menyimpan status limiter di RAM aplikasi. Jika server backend dinaikkan skalanya menjadi multi-instance (berjalan di balik Load Balancer) atau container di-restart, data rate limit klien akan ter-reset.

- Tugas:

1. Tulis middleware rate limiter dinamis baru bernama `RedisRateLimiterMiddleware()` di dalam `middleware/ratelimit.go`.
2. Setiap request masuk, ambil alamat IP klien dan susun key Redis: `rateLimit:<client_ip>`.
3. Gunakan perintah Redis `INCR` pada key tersebut untuk menambah jumlah hit request.
4. Jika nilai hit bernilai 1 (baru dibuat), set durasi kedaluwarsa key menggunakan `EXPIRE` selama 60 detik (1 menit).
5. Jika nilai hit melebihi batas (misal: maksimum 20 request per menit), gagalkan request dan kembalikan status HTTP 429 Too Many Requests.
6. Metode ini menjamin rate limiting sinkron di seluruh instance backend karena status disimpan di satu Redis server yang sama.

Tantangan 2: Docker Compose Healthcheck & Dependency Order

Saat menjalankan `docker compose up`, container aplikasi Go (`web`) sering crash saat startup karena container database (`db`) membutuhkan waktu sekitar 5–10 detik untuk menginisialisasi sistem internalnya sebelum siap menerima koneksi socket, meskipun Anda sudah menyetel `depends_on: - db` (karena dependensi default hanya mendeteksi container database *menyala*, bukan *siap* menerima koneksi).

- Tugas:

1. Modifikasi file `docker-compose.yml` Anda.
2. Tambahkan instruksi `healthcheck` di bawah service `db` :

```
healthcheck:
  test: ["CMD-SHELL", "pg_isready -U postgres -d eticketdb"]
```

```
interval: 5s
timeout: 5s
retries: 5
```

3. Ubah konfigurasi `depends_on` pada service `web` agar mendeteksi status kesehatan database:

```
depends_on:
  db:
    condition: service_healthy
  cache:
    condition: service_started
```

4. Uji coba dengan mematikan compose lalu menghidupkannya kembali, dan pastikan container aplikasi Go menunda startup-nya sampai database PostgreSQL melaporkan status sehat (*healthy*).